

Y E H I O R T E C H

MédiHelm

PROMPT CURSOR — IMPLÉMENTATION SCAN CODE-BARRES v1.0

Prompt de production prêt à coller dans Cursor AI

Référence

MH-SCAN-CURSOR-2025-v1.0

Périmètre

Scan EAN-13 · DataMatrix GS1 · Caméra PWA · Lecteur USB

Modules

M01 Stock · M02 POS · M01 Inventaire · M18 Vérification DPMED

Priorité

HAUTE — À implémenter après backend 16 modules

Usage

Coller dans Cursor — mode Agent — sélectionner toute la base de code

Responsable

Dawes — Fondateur & Lead Developer, YEHI OR Tech

Document confidentiel — YEHI OR Tech © 2025

Instructions d'utilisation

1. Ouvrir Cursor AI dans le workspace MédiHelm (medihelm-api + medihelm-web).
2. Mode : Agent (Ctrl+Shift+P → 'Open Agent')
3. Sélectionner les dossiers pertinents dans le contexte :
 - medihelm-api/src/modules/ (modules NestJS existants)
 - medihelm-api/src/prisma/schema.prisma
 - medihelm-web/src/components/ (composants React existants)
 - medihelm-web/src/hooks/
4. Coller le prompt complet ci-dessous.
5. Laisser l'agent implémenter — valider les migrations Prisma manuellement avec : `npx prisma migrate dev --name add_scan_fields`

Le backend des 16 modules est terminé. Ne pas modifier les modules existants sauf pour ajouter les champs Prisma et les méthodes de scan.
Toujours vérifier les migrations avant de les appliquer en base.

PROMPT CURSOR — COPIER TOUT CE QUI SUIT

Tu travailles sur MédiHelm Pro — un ERP SaaS pharmaceutique pour officines béninoises (YEHI OR Tech, Parakou, Bénin). Stack : Next.js 14 App Router + NestJS + PostgreSQL + Prisma + Supabase + Tailwind CSS.

Le backend des 16 modules est terminé et fonctionnel. Ta mission est d'implémenter le scan de code-barres médicaments de bout en bout.

CONTEXTE MÉTIER

Les médicaments au Bénin portent deux types de codes principaux :

- EAN-13 : 13 chiffres — présent sur quasi tous les médicaments importés
- DataMatrix GS1 : encode GTIN + lot + date expiration via Application Identifiers : (01)GTIN-14 (10)lot (17)YYMMDD (21)série

Le scan intervient dans 4 contextes :

1. POS_VENTE : ajout direct au panier avec lot FEFO
2. RECEPTION : pré-remplissage formulaire réception lot
3. INVENTAIRE : identification lot + saisie quantité réelle
4. VERIFICATION : interrogation alerte DPMED sur le lot

Deux types de matériel :

- Lecteur USB code-barres : envoie les chars comme un clavier, se termine par Enter. Vitesse < 50ms entre caractères.
- Caméra smartphone/tablette : via API navigator.mediaDevices

TÂCHE 1 — MIGRATIONS PRISMA

Ajoute ces champs dans schema.prisma :

```
model Medicament {
  // Ajouter :
  gtin          String?    // GTIN-14 GS1
  codeAMM        String?    // Numéro AMM DPMED Bénin
  @@index([pharmacieId, codeBarres])
  @@index([pharmacieId, gtin])
}

model Lot {
  // Ajouter :
  gtinLot        String?    // GTIN du lot
  numeroSerie    String?    // Série GS1 AI-21
  rawBarcode      String?    // Code brut original scanné
}
```

Génère la migration : `npx prisma migrate dev --name add_scan_fields`

TÂCHE 2 — UTILITAIRE GS1 (PARTAGÉ FRONT + BACK)

Crée utils/gslParser.ts (utilisable côté client ET serveur) :

```
export interface GS1Data {
  gtin?      : string;
  ean13?     : string;
  lot?       : string;
  expiry?    : string;    // YYMMDD brut
  expiryDate? : Date;
  serial?    : string;
  rawCode?   : string;
}

export function parseGS1(raw: string): GS1Data {
  // 1. EAN-13 : /^d{13}$/ → gtin = '0' + raw, ean13 = raw
  // 2. DataMatrix avec parenthèses : (01)xxx(10)yyy via regex
  //    /(\\d{2,4}\\))([^(]*)/g
  //    AI 01 → gtin, AI 10 → lot, AI 17 → expiry, AI 21 → serial
  // 3. Format compact sans parenthèses : commence par '01' + 14 chiffres
  // 4. Autres : retourner { rawCode: raw } pour résolution serveur
  //
  // Pour AI 17 : YYMMDD, si DD='00' → dernier jour du mois
  // expiryDate = new Date('20YY-MM-DD')
}
```

TÂCHE 3 — HOOK useBarcodeInput (FRONTEND)

Crée hooks/useBarcodeInput.ts :

Écoute window keydown en permanence.
 Accumule les caractères dans un buffer.
 Si délai entre deux chars < 50ms → lecteur USB → accumuler.
 Si délai > 300ms → saisie humaine → vider le buffer.
 Sur Enter : si buffer.length >= 6 → appeler onBarcode(buffer).
 Prop 'enabled' pour activer/désactiver selon la page active.

```
interface Options {
  onBarcode      : (code: string) => void
  minLength?     : number    // défaut 6
  speedThreshold? : number    // ms défaut 50
  enabled?       : boolean    // défaut true
}
```

TÂCHE 4 — COMPOSANT BarcodeScanner (CAMÉRA PWA)

Installation : npm install @zxing/browser @zxing/library

Crée components/scan/BarcodeScanner.tsx :

- useRef<HTMLVideoElement> pour le flux caméra
- BrowserMultiFormatReader de @zxing/browser

- decodeFromConstraints({ video: { facingMode: 'environment' } })
- Prop 'active: boolean' - démarre/arrête le scan
- Prop 'onScan(raw: string)' - callback quand code détecté
- Prop 'onError(err: Error)' - si caméra refusée
- Un scan réussi appelle reader.reset() pour arrêter
- Overlay teal (cadre de visée) centré sur la vidéo
- Ce composant doit être importé avec next/dynamic, SSR: false

TÂCHE 5 - ENDPOINT POST /scan (NESTJS BACKEND)

Crée le module scan/ avec :

```
scan.controller.ts :
  @Post() @UseGuards(JwtAuthGuard)
  scan(@Body() dto: ScanDto, @CurrentUser() user: JwtPayload)

scan.dto.ts :
  code: string (IsString, IsNotEmpty)
  contexte: ContexteScan (IsEnum)

enum ContexteScan {
  POS_VENTE = 'POS_VENTE'
  RECEPTION = 'RECEPTION'
  INVENTAIRE = 'INVENTAIRE'
  VERIFICATION = 'VERIFICATION'
}

scan-resolver.service.ts - logique complète :

async resoudre(rawCode, pharmacieId, contexte):

1. parseGS1(rawCode) → gs1Data

2. Chercher médicament dans Prisma :
  WHERE pharmacieId = pharmacieId
  AND (codeBarres = gs1.ean13 OR gtin = gs1.gtin
      OR codeBarres = gs1.rawCode)
  AND actif = true

3. Si non trouvé :
  → Chercher dans FicheDCI (base nationale) par dci ou gtin
  → Si trouvé : { statut: 'NOUVEAU', suggererAjout: true, gs1Data }
  → Sinon : { statut: 'INCONNU', rawCode }

4. Si lot GS1 fourni (gs1.lot) : vérifier AlerteDPMED :
  WHERE numerosLotConcernes HAS gs1.lot AND statut = EN_DIFFUSION
  → Si trouvé : { statut: 'ALERTE', médicament, lot: gs1.lot, alerte }

5. Switch contexte :

  POS_VENTE :
    → Lot.findFirst WHERE médicamentId, pharmacieId (via médicament),
       quantite > 0, dateExpiration > now()
```

```

ORDER BY dateExpiration ASC (FEFO)
→ Si aucun lot : { statut: 'RUPTURE', médicament }
→ Si périmé (dateExpiration < now) : { statut: 'PERIME', médicament, lot }
→ Sinon : { statut: 'TROUVE', médicament, lot, action: 'AJOUTER_PANIER' }

RECEPTION :
→ { statut: 'TROUVE', médicament, gs1Data, action: 'PREFILL_RECEPTION' }

INVENTAIRE :
→ Lot.findFirst par numeroLot = gs1.lot si disponible
→ Sinon premier lot FEFO
→ { statut: 'TROUVE', médicament, lot, action: 'SAISIR_QUANTITE' }

VERIFICATION :
→ Vérifier AlerteDPMED + statut lot
→ { statut: 'TROUVE', médicament, gs1Data, statutDPMED }

```

TÂCHE 6 – INTÉGRATION POS (FRONTEND)

Dans la page POS existante (M02) :

- Ajouter useBarcodeInput({ onBarcode: handleScan, enabled: true })
→ Le hook est actif en permanence sur la page POS
- Ajouter un bouton 'Scanner' qui active le BarcodeScanner
→ Importé via next/dynamic avec ssr: false
- Implémenter handleScan(rawCode: string) :
→ parseGS1(rawCode) côté client
→ POST /api/scan { code: rawCode, contexte: 'POS_VENTE' }
→ Switch sur result.statut :
TROUVE → ajouterAuPanier(result.medicament, result.lot)
ALERTE → afficher AlerteModal (rouge, acquittement obligatoire)
si alerte.niveauUrgence === 'URGENCE_IMMEDIATE' : bloquer
PERIME → toast rouge 'Lot périmé depuis le JJ/MM/AAAA'
RUPTURE → toast orange 'Rupture – NomMédicament'
NOUVEAU → drawer 'Ajouter au catalogue ?' avec données GS1
INCONNU → toast gris 'Code non reconnu' + focus champ recherche

TÂCHE 7 – INTÉGRATION RÉCEPTION STOCK (FRONTEND)

Dans le formulaire de réception stock (M01) :

- Ajouter bouton 'Scanner la boîte reçue'
- Appeler POST /api/scan { contexte: 'RECEPTION' }
- Sur TROUVE ou NOUVEAU :
→ Pré-remplir automatiquement :
- Médicament (select) → result.medicament
- Numéro lot → result.gs1Data.lot
- Date expiration → result.gs1Data.expiryDate
- GTIN → result.gs1Data.gtin (champ caché pour stockage)

- Laisser Quantité et Prix achat à saisir manuellement
- 4. Sur NOUVEAU : proposer création médicament avant la réception

TÂCHE 8 – INTÉGRATION INVENTAIRE (FRONTEND)

Dans l'écran de session d'inventaire (M01 InventaireSession) :

1. Ajouter useBarcodeInput({ onBarcode: handleScan })
2. Appeler POST /api/scan { contexte: 'INVENTAIRE' }
3. Sur TROUVE :
 - Afficher : nom médicament, numéro lot, quantité système
 - Focus automatique sur champ 'Quantité comptée'
 - Après saisie + Enter : PATCH /stock/inventaire/:sessionId/lots/:lotId
body: { quantiteReelle: number }
 - Réinitialiser pour prochain scan
4. Sur INCONNU : afficher 'Lot non référencé – vérifier ou saisir manuellement'

TÂCHE 9 – MODE OFFLINE

Crée scan/scan-offline.service.ts :

- Même interface que ScanResolverService
- Interroge SQLite (Prisma client SQLite – déjà configuré)
- Tables disponibles offline : médicaments, lots
- Les alertes DPMED NE sont PAS vérifiées offline (pas de connexion)
- Ajouter une note visuelle 'Mode hors ligne – alertes DPMED non vérifiées'

Dans le composant POS, sélectionner le service selon navigator.onLine.

CONTRAINTES À RESPECTER ABSOLUMENT

- NE PAS modifier les modules POS, Stock, Patients existants
sauf pour y brancher le hook useBarcodeInput et le bouton scan.
- POST /scan doit répondre en < 300ms – utiliser les index Prisma.
- Le BarcodeScanner DOIT être importé avec next/dynamic, ssr: false
(Leaflet et ZXing ne sont pas SSR-compatibles).
- Toutes les réponses de POST /scan suivent le même format :
{ statut: string, médicament?, lot?, gslData?, action?, alerte? }
- Le hook useBarcodeInput utilise window.addEventListener – ne pas
l'appeler dans un composant Server Component Next.js.
- Stocker rawBarcode dans Lot à chaque scan pour audit.
- Ne pas créer de dépendance à un service tiers pour la résolution
des codes – tout se fait en base Prisma locale.

ORDRE D'IMPLÉMENTATION RECOMMANDÉ

1. utils/gslParser.ts + tests unitaires
2. Migration Prisma (champs + index)
3. hooks/useBarcodeInput.ts

```

4. components/scan/BarcodeScanner.tsx
5. scan/ module NestJS (controller + dto + service)
6. Intégration POS (M02)
7. Intégration réception stock (M01)
8. Intégration inventaire (M01)
9. scan-offline.service.ts
10. Tests : CA-01 à CA-10 (voir CDC)

```

NOTES COMPLÉMENTAIRES POUR CURSOR

Arborescence des fichiers à créer

```

medihelm-api/
  src/
    modules/
      scan/
        scan.module.ts
        scan.controller.ts
        scan.dto.ts
        scan-resolver.service.ts
        scan-offline.service.ts
    utils/
      gs1Parser.ts
    prisma/
      migrations/
        YYYYMMDD_add_scan_fields/
          migration.sql

medihelm-web/
  src/
    hooks/
      useBarcodeInput.ts
    components/
      scan/
        BarcodeScanner.tsx
        ScanResultToast.tsx // composant UI feedback
        AlerteDPMEDModal.tsx // modal alerte rouge

```

Tests unitaires minimaux à écrire

```

// gs1Parser.test.ts
describe('parseGS1', () => {
  test('EAN-13 simple', () => {
    const r = parseGS1('3560070162439');
    expect(r.ean13).toBe('3560070162439');
    expect(r.gtin).toBe('03560070162439');
  });

  test('DataMatrix GS1 complet', () => {

```

```
const r = parseGS1('(01)03560070162439(10)BN2024-441A(17)260831');
expect(r.gtin).toBe('03560070162439');
expect(r.lot).toBe('BN2024-441A');
expect(r.expiry).toBe('260831');
expect(r.expiryDate).toEqual(new Date('2026-08-31'));
});

test('AI 17 avec DD=00 → dernier jour du mois', () => {
  const r = parseGS1('(01)xxx(17)260800');
  expect(r.expiryDate?.getDate()).toBe(31); // 31 août 2026
});

test('Code inconnu → rawCode préservé', () => {
  const r = parseGS1('ABC-12345-INTERNAL');
  expect(r.rawCode).toBe('ABC-12345-INTERNAL');
  expect(r.gtin).toBeUndefined();
});
});
```